1. Title: Machine Learning for Web Attack Detection Using Web Activity Logs

Injection detection (OWASP Top 10:2025, A05) on OWASP Juice Shop · Group 2 · Master Intelligence Artificielle et Cybersécurité, USMS Béni Mellal

Members: Houssayni Maryame, Ait Kaddour Salma, Adoch Issam, Khannafour Mouad, Kenzi Oussama · Group leader: Ait Kaddour Salma

2. Research question and hypothesis

Question. Can a model trained only on a public HTTP request dataset (CSIC 2010, filtered for injection) detect injection attacks (SQL injection, optionally XSS) against a different application, OWASP Juice Shop, whose requests we collect ourselves? Which model, classical or pre-trained Transformer, generalizes best?

Hypotheses. H1: every model will score lower on the Juice Shop test set than on the CSIC validation split, because of the domain gap (HTML forms vs JSON API). **H2:** DistilBERT will reach a higher injection Recall than the hand-crafted Random Forest, since self-attention can combine a quote, an SQL keyword and a comment marker without manual features; but the character n-gram Logistic Regression may stay competitive, because DistilBERT's natural-language vocabulary splits payloads into many fragments. No performance level is assumed in advance.

3. Dataset

	Name / link	Size	Class balance
Training (main)	HTTP CSIC 2010 - tic.itefi.csic.es/dataset (Kaggle mirror: kaggle.com/datasets/ispangler/csic-2010-web-application-attacks)	~36,000 normal (train) + ~36,000 normal (test) + >25,000 anomalous HTTP requests	About 74 % normal / 26 % anomalous before filtering. We keep only anomalous requests with injection markers (SQL keywords, special characters, script tags), so the final count and ratio will be documented; class weighting or undersampling will handle the imbalance.
Backup	SQL Injection Dataset (Kaggle) - kaggle.com/datasets/sajid576/sql-injection-dataset	~30,900 benign and malicious queries	Roughly balanced (to be verified). Raw SQL queries, not HTTP requests: used only to enrich the Transformer if useful, or as fallback training data.
Test (ours)	Juice Shop requests collected in our local lab (Nginx or mitmproxy logs)	Fixed in week 3 and documented	Labelled 0 = normal / 1 = injection by the generator script, scenario kept, sample reviewed by hand. Held out: never used for training or tuning.

4. Method (baseline first, then main model)

Common preprocessing for both datasets: method + path + parameters + body, URL-decoded, lower-cased, JSON flattened to text; source-specific fields removed (host, cookies, session IDs, User-Agent, timestamp) to avoid shortcut learning; duplicates removed.

- **Baseline - Logistic Regression:** TF-IDF on character n-grams + a few manual features.
- **Second classical model - Random Forest:** manual features (length, special-character count, SQL keywords, encoded-character rate, number of parameters).
- **Main model - DistilBERT** (Hugging Face Transformers, PyTorch), fine-tuned on filtered CSIC 2010 with a 2-class head, 128-256 tokens, on free Google Colab GPU. A publicly available SQL-injection DistilBERT may be run as an unadapted reference (its published scores concern another dataset and will not be reported as ours).

5. Evaluation

- **Metrics:** Precision, Recall, F1-score, confusion matrix and Accuracy. Accuracy is never reported alone; Recall of the injection class and F1 are the priority.
- **Split:** CSIC is split ~80 % train / ~20 % validation (hyper-parameters and decision threshold tuned on validation only). After freezing the models, a single final evaluation is run on the Juice Shop lab dataset, identical for the three models. Validation scores are secondary and serve to measure the domain gap.
- **Live test idea:** a small script tails the proxy log of the lab while we replay fresh traffic (new SQL payloads and variants - case, comments, URL encoding - plus normal browsing scripts, not present in the test set). Each request is scored as it arrives and logged with its latency. We report detection rate, false alarms and delay. It is alert-only, no blocking.
- **Error analysis:** missed request types, format differences, effect of encoding and JSON bodies.

6. MLOps plan

Stage	Plan (free tools, local lab scale)
Serving	Best model + preprocessing packaged in a Docker container exposing a FastAPI `/predict` endpoint (input: normalized request; output: label + score). The log-tailing script from the live test calls it.
Monitoring	Log scores, predicted-injection rate, latency; track input drift (request length, special-character rate, share of JSON bodies) against the training data; periodically score a labelled replay to estimate Recall on fresh attacks.

Stage	Plan (free tools, local lab scale)
Retraining	Data, code and models versioned (Git, MLflow or DVC). Retrain if drift is flagged or replay Recall falls below a threshold set after the first results. New lab traffic for retraining is collected separately; the original test set stays untouched for comparisons.

7. Main risks and ethics

Risk	Mitigation
Domain gap CSIC vs Juice Shop	Common representation, error analysis, result reported as is
CSIC anomalies are not all injections	Rule-based filter + manual check of a sample
Lab dataset small, uniform or noisy labels	Vary scenarios and payloads, review a sample, document real size
Artifacts learned instead of attacks (e.g. tool User-Agent)	Remove source-specific fields, randomize generators
Information leak into the test set	Lab data never used for training or tuning
Transformer slow or GPU unavailable	Small DistilBERT, short sequences, Colab, CPU as fallback

Confirmation: all attacks (including any sqlmap use) are run **only in our own isolated local lab** (Docker, OWASP Juice Shop on localhost:3000, no Internet exposure). No third-party system is targeted.